MedAI Frontend — Project Status Report

> **Date:** 8 February 2026
> **Scope:** Frontend (Next.js 14 / React 18 / Tailwind CSS)
> **Build status:** ✅ Production build passes — 9 routes, 0 TypeScript errors

---

Table of Contents

- MedAI Frontend — Project Status Report
  - Table of Contents
  - 1. Executive Summary
  - 2. Technology Stack
  - 3. Implemented Features & Current State
    - 3.1 Pages & Routes
    - 3.2 Component Library
    - 3.3 API Client Layer
    - 3.4 Authentication System
    - 3.5 Accessibility (a11y)
    - 3.6 Design System & Theming
    - 3.7 Error Handling
    - 3.8 Mock Data / Graceful Degradation
  - 4. Next Steps — Features to Implement
    - 4.1 Critical (MVP Blockers)
    - 4.2 High Priority (Usable Product)
    - 4.3 Medium Priority (Polish)
    - 4.4 Low Priority (Nice-to-Have)
  - 5. Known Problems, Errors & Open Questions
    - 5.1 Bugs & Technical Debt
    - 5.2 Architecture Concerns
    - 5.3 UX / Design Gaps
    - 5.4 Open Questions for Brainstorming
  - 6. File-by-File Status Matrix
    - Pages
    - Components
    - Library
  - 7. Dead / Unused Code Inventory
  - 8. Hardcoded Values Inventory

---

1. Executive Summary

The frontend is a **functional prototype** with a complete page structure, working navigation, a dual-mode API client (live backend + mock fallback), a JWT-ready auth system, and solid accessibility foundations. The core user journey — *landing → co-pilot chat → case report → doctor approval* — is navigable end-to-end.

However, several critical features remain **stubs, placeholders, or partially implemented**: file uploads are accepted in the UI but silently discarded, the image viewer renders an emoji instead of actual DICOM/medical images, the admin dashboard has two placeholder sections, and the auth system lacks token refresh and session validation. The project is at roughly **65–70% completion** toward a usable demonstration product.

---

2. Technology Stack

Layer	Technology	Version
Framework	Next.js (App Router)	14.2.20
UI Library	React	18.3.1
Language	TypeScript	5.6 (strict mode)
Styling	Tailwind CSS	3.4.15
Animations	framer-motion	11.12.0
Icons	lucide-react	0.460.0
Utility	clsx + tailwind-merge	2.1 / 2.6
Theme	Custom provider (no next-themes dep)	–
Backend target	FastAPI	localhost:8000/api/v1

Dependency count: 7 production + 8 dev — very lean.

---

3. Implemented Features & Current State

3.1 Pages & Routes

Route	Page	State	Description
`/`	Landing	✅ Complete	Hero, Features, Architecture, Stats, CTA, Footer. All content hardcoded.
`/agent`	Co-Pilot Chat	✅ Functional	Dual-mode: real `analyzeCase` API call when backend online, timed mock simulation when offline. Live/Mock indicator.
`/case/[id]`	Case Report	✅ Functional	Fetches report from API, maps to frontend types, falls back to mock. Full approval workflow wired to `approveReport` endpoint.
`/timeline/[patientId]`	Patient Timeline	✅ Functional	Fetches timeline + patient from API, falls back to mock. Filter by event type, grouped by year.

Route	Page	State	Description
`/patients`	Patient List	✅ Functional	Lists patients from API, search by name/ID/MRN, create-patient modal. **No mock fallback — shows error if backend is down.**
`/admin`	Admin Dashboard	⚠ Partial	Health check + patient count sections are live. **Reports section and Usage Statistics section are stubs** with placeholder text. Role-gated to `admin`.
`/auth/login`	Login	✅ Complete	Email + password form, error display, redirect on success.
`/auth/register`	Register	✅ Complete	Name + email + password form, minLength validation, redirect on success.
`/not-found`	404	✅ Complete	Custom 404 with link to home.
`/error`	Error	✅ Complete	Global error boundary page with reset button.

3.2 Component Library

Agent components (4):

Component	State	Notes
`ChatArea`	✅ Complete	Auto-scroll, empty state with 4 suggested prompts, prompt click handler.
`ChatInput`	✅ Complete	Auto-resize textarea, file attach menu (image/audio/doc), Enter to send. Voice button disabled ("coming soon").
`ChatMessage`	✅ Complete	Safe inline rendering (no `dangerouslySetInnerHTML`), tool result display, citation badges, attachment chips, timestamps with `<time>`.
`CitationsSidebar`	✅ Complete	4-tab filter (Evidence/Findings/History/Guidelines), expand/collapse citations, confidence badges, cross-reference summary.

Case components (4):

Component	State	Notes

Component	State	Notes
`ImageViewer`	⚠️ Placeholder	Accepts `imageUrl` prop and has **real heatmap generation code** (Gaussian blobs, Jet colormap in canvas), zoom controls, opacity slider, and bounding box overlay — but the actual `<img>` element is **never rendered**. Instead, displays an emoji 🩻 with text "CXR Image".
`FindingsPanel`	✅ Complete	Findings list with severity/confidence badges, differential diagnoses, recommended follow-up.
`ReasoningTrace`	✅ Complete	Step-by-step expandable reasoning with framer-motion animations, expand all/collapse all.
`ApprovalBar`	✅ Complete	Approve/reject/edit workflow, notes textarea, loading states, post-decision confirmation.

Shared components (7):

Component	State	Notes
`ThemeToggle`	✅ Complete	3-way toggle (light/dark/system), shows resolved theme for system mode.
`AgentStatusIndicator`	✅ Complete	10 agent states with animated icons, pulse ring on active states.
`ErrorBoundary`	✅ Complete	Class-based error boundary with reset, custom label, optional fallback UI. Wraps root layout.
`ProtectedRoute`	✅ Complete	Auth guard, role-based access, redirect to login, loading spinner.
`ConfidenceBadge`	✅ Complete	Color-coded percentage badge (emerald/amber/rose).
`SeverityBadge`	✅ Complete	6-level severity with icons (low → critical).
`LoadingAnimation`	✅ Complete	3 variants (dots/pulse/orbital), used across all loading.tsx files.

Landing components (5): Hero, Features, Architecture, Stats, CTA — all ✅ complete with hardcoded content.

Layout components (2): Navbar (✅ complete, auth-aware user menu), Footer (✅ complete).

Unused component: `ToolProgress` — exists but is not imported by any page. `ChatMessage` has its own inline tool result rendering instead.

3.3 API Client Layer

The API layer was refactored into a **class-based singleton** with automatic JWT auth header injection:

```
src/lib/api/
├── client.ts   ← ApiClient class (singleton: apiClient)
├── types.ts    ← All request/response TypeScript interfaces
└── index.ts    ← Barrel re-export
```

Covered endpoints (11):

Method	Endpoint	Client Method	Used By
GET	`/health`	`checkHealth()`	Admin page
GET	`/patients`	`listPatients()`	Patients page, Admin page
GET	`/patients/:id`	`getPatient()`	Case page, Timeline page
POST	`/patients`	`createPatient()`	Patients page
GET	`/patients/:id/timeline`	`getPatientTimeline()`	Timeline page
GET	`/patients/:id/reports`	`getPatientReports()`	*(available but unused)*
GET	`/cases/reports/:id`	`getReport()`	Case page
POST	`/cases/analyze`	`analyzeCase()`	Agent page
POST	`/cases/approve`	`approveReport()`	Case page
POST	`/auth/login`	`login()`	Auth login page
POST	`/auth/register`	`register()`	Auth register page
GET	`/auth/me`	`getMe()`	*(available but unused)*

Hooks layer (`src/lib/hooks.ts`): 7 hooks defined, but only `usePatients()`, `usePatient()`, `usePatientTimeline()`, and `useReport()` are actively used. The `useCaseAnalysis()` and `useReportApproval()` hooks exist but are bypassed — pages call `apiClient` directly.

3.4 Authentication System

Feature	State	Details
AuthProvider context	✅ Implemented	Wraps entire app in root layout.
Login flow	✅ Implemented	Calls `apiClient.login()`, stores JWT + user in localStorage.
Register flow	✅ Implemented	Calls `apiClient.register()`, stores JWT + user in localStorage.
Logout	✅ Implemented	Clears localStorage, resets user state.

Feature	State	Details
Auto-hydration on mount	✅ Implemented	Reads user from localStorage on page load.
Auth header injection	✅ Implemented	`ApiClient.request()` auto-injects `Authorization: Bearer <token>`.
Protected routes	✅ Implemented	`ProtectedRoute` wrapper, used by admin layout.
Role-based access	✅ Implemented	`requiredRole` prop on ProtectedRoute.
Navbar user menu	✅ Implemented	Shows user name/email, admin link (if admin), logout button.
Token refresh	❌ Not implemented	No refresh token flow — token persists until manual logout or expiry.
Session validation (getMe)	❌ Not implemented	`getMe()` method exists but is never called on mount to validate token.
Password reset	❌ Not implemented	No forgot-password flow.

3.5 Accessibility (a11y)

The project has **strong accessibility foundations**:

- ✅ Skip-to-content link (`<a href="#main-content" className="sr-only focus:not-sr-only">`)
- ✅ `<html lang="en">`
- ✅ `role` attributes on interactive widgets: `tablist`, `tab`, `tabpanel`, `status`, `alert`, `menu`, `menuitem`, `radiogroup`, `radio`, `region`, `list`, `button`, `img`, `note`
- ✅ `aria-*` attributes: `aria-label`, `aria-expanded`, `aria-selected`, `aria-pressed`, `aria-checked`, `aria-controls`, `aria-live="polite"`, `aria-atomic`, `aria-hidden`, `aria-haspopup`
- ✅ `<time dateTime>` for all date/time displays
- ✅ `<label htmlFor>` + `id` on all form inputs
- ✅ `autoComplete` attributes on auth form fields
- ✅ `sr-only` hidden labels for inputs without visible labels
- ✅ `prefers-reduced-motion` CSS media query (disables animations)
- ✅ No `dangerouslySetInnerHTML` — safe React element rendering

Minor gaps:

- Some sidebar toggle buttons use `title` but not `aria-label` (agent page panel toggles)
- No `aria-describedby` linking error messages to form fields
- No focus trap in modal dialogs (create patient modal, attach file dropdown)

3.6 Design System & Theming

Feature	State
Custom color palette (`brand`, `surface`, `accent-*`)	✅ Implemented
Dark/light/system theme toggle	✅ Implemented
Glass morphism effects (`glass-card`)	✅ Implemented
Neumorphic shadows (`neo-shadow`)	✅ Implemented
Custom animations (7 keyframes)	✅ Implemented
Custom scrollbar styling	✅ Implemented
Responsive design (mobile breakpoints)	✅ Implemented
Custom fonts (Inter + JetBrains Mono)	✅ Implemented
Gradient text utility	✅ Implemented

3.7 Error Handling

Layer	Mechanism	State
Global page errors	`src/app/error.tsx` (Next.js convention)	✅
Component render errors	`ErrorBoundary` wrapping `{children}` in layout	✅
API fetch errors	Try/catch with mock fallback or error state UI	✅
Custom error class	`ApiError` with `status`, `message`, `details`	✅
Form errors	Inline error alerts with icon + retry	✅
404	Custom `not-found.tsx`	✅

3.8 Mock Data / Graceful Degradation

The application uses a **dual-mode** pattern:

1. On mount, checks `apiClient.isBackendAvailable()` (or tries the API call directly)
2. If backend responds → uses real data, shows "Live" indicator
3. If backend unreachable → falls back to `src/lib/mock-data.ts`, shows "Mock"/"Demo" indicator

Exception: The `/patients` page does **not** have a mock fallback. If the backend is down, it displays an error with a retry button.

The mock dataset represents a coherent clinical scenario: a patient named "Olena Kovalenko" (PT-12345) with community-acquired pneumonia, including 7 timeline events, 3 findings with bounding boxes, 6 reasoning steps, 6 citations, and a full AI report.

---

4. Next Steps — Features to Implement

4.1 Critical (MVP Blockers)

These features are necessary for the product to function as a minimally viable demonstration:

#	Feature	Why It's Critical	Files Affected
1	**Render actual medical images in ImageViewer**	The image viewer shows an emoji placeholder 🩻 instead of actual images. The heatmap overlay code exists but operates on nothing. Without real image rendering, the case view — the most important clinical screen — is not demonstrable.	`ImageViewer.tsx`
2	**Wire file uploads to the backend**	ChatInput accepts images/audio/documents, but `handleSend` in agent page silently discards the `_attachments` parameter. Users can select files but nothing happens with them. The `analyzeCase` API accepts `image_urls` and `audio_urls` — these need to be populated, possibly via a file upload endpoint or presigned URLs.	`agent/page.tsx`, `api/client.ts`
3	**Call `getMe()` on mount to validate stored token**	Currently, AuthProvider hydrates the user from localStorage without checking if the JWT is still valid. If the token expired, the user sees a logged-in UI but all API calls fail with 401. Should call `apiClient.getMe()` on mount and clear stale sessions.	`AuthProvider.tsx`
4	**Dynamic patient context in Co-Pilot**	The agent page always shows `mockPatient` ("Olena Kovalenko") in the top bar and sends `mockPatient.id` to the API regardless of which patient the doctor is working with. The patient should either be selectable or passed via URL params.	`agent/page.tsx`

4.2 High Priority (Usable Product)

#	Feature	Description
5	**Patient detail page**	A dedicated `/patients/[id]` page showing patient demographics, their reports list (`getPatientReports`), and a link to their timeline. Currently, clicking a patient in the list goes straight to timeline.
6	**Populate admin Reports section**	The admin dashboard's "Report Audit" section is a placeholder. Should fetch and display recent reports, approval statuses, and aggregate statistics. Requires either a new backend endpoint or client-side aggregation from `getPatientReports`.
7	**Populate admin Usage Statistics section**	The "Usage Statistics" section is a placeholder. Should show API call counts, average response times, and model usage. Requires backend analytics endpoint.

#	Feature	Description
8	**Toast / notification system**	Currently, success/error feedback is shown only inline. A global toast system would improve UX for actions like "Report approved", "Patient created", "Login successful".
9	**Loading states on navigation**	Next.js `loading.tsx` files exist, but client-side navigation between pages doesn't always trigger them. Consider a top-of-page progress bar (e.g., NProgress).
10	**Handle backend auth endpoints not existing**	If the backend hasn't implemented `/auth/login`, `/auth/register`, `/auth/me`, the login/register pages will show a generic "API error 404" or network error. Should show a user-friendly message like "Authentication is not available on this server".

4.3 Medium Priority (Polish)

#	Feature	Description
11	**Token refresh mechanism**	Implement refresh token flow (silent refresh or redirect to login on 401).
12	**Focus trap in modals**	The create-patient modal and file-attach dropdown don't trap focus. Tab can escape to elements behind the overlay.
13	**Keyboard shortcuts**	Useful shortcuts: `Ctrl+/` to focus chat input, `Esc` to close modals/sidebars.
14	**Use `DEFAULTS` and `API_PATHS` constants**	`DEFAULTS.demoPatientId` and `API_PATHS` are defined in `constants.ts` but never imported. Navbar hardcodes `"PT-12345"` and the API client hardcodes endpoint paths.
15	**Streaming responses**	Agent page sends a request and waits for the full response. For long analyses, streaming the response (SSE or WebSocket) would provide real-time feedback instead of a spinner.
16	**Pagination on patient list**	Currently loads all patients at once. For large datasets, needs server-side pagination with offset/limit params.
17	**Date localization**	`formatDate()` is hardcoded to `en-US`. Should use the browser's locale.
18	**SEO / Open Graph meta tags**	Only basic `title` and `description` in metadata. Missing OG image, OG type, Twitter card.

4.4 Low Priority (Nice-to-Have)

#	Feature	Description
19	**Audit log**	Record who approved/rejected each report and when. (Likely backend-driven.)

#	Feature	Description
20	**Export report as PDF**	Allow doctors to download a formatted PDF of the AI report.
21	**Multi-language support (i18n)**	Currently English-only.
22	**E2E tests**	No tests exist. Playwright or Cypress for critical flows (login → chat → case view → approve).
23	**Storybook**	Component catalog for the custom component library.
24	**PWA support**	Manifest + service worker for offline-capable demo.

5. Known Problems, Errors & Open Questions

5.1 Bugs & Technical Debt

#	Problem	Severity	Details
B1	**ImageViewer never renders an `<img>` element**	🔴 High	The component accepts `imageUrl` as a prop but renders a `<div>` with an emoji instead of an actual image. The canvas-based heatmap overlay, zoom controls, and bounding box system are all built but operate on nothing. This is the single most visible gap in the product.
B2	**File attachments are silently discarded**	🔴 High	`ChatInput` collects files into an array, passes them to `onSend(text, attachments)`, but `handleSend` in `agent/page.tsx` names the parameter `_attachments` (underscore = intentionally unused) and never processes them. No upload endpoint exists either.
B3	**Mock simulation timeouts not cleaned up on unmount**	🟡 Medium	In `agent/page.tsx`, the mock fallback uses multiple `setTimeout` calls (lines 80–90) that are not cancelled if the component unmounts during the simulation. This can cause "setState on unmounted component" warnings. Should use a ref flag or `AbortController`.
B4	**Patients page has no mock fallback**	🟡 Medium	Unlike `/agent`, `/case`, and `/timeline`, the `/patients` page has no graceful degradation to mock data when the backend is unreachable. It shows an error message instead. This is inconsistent with the rest of the app.
B5	**`useAsync` deps array uses eslint-disable**	🟡 Medium	The generic `useAsync` hook in `hooks.ts` accepts a `deps` array and passes it to `useCallback`, with `// eslint-disable-next-line react-hooks/exhaustive-deps`. While functional, this suppresses a legitimate linting rule and could mask stale-closure bugs.

#	Problem	Severity	Details
B6	**Case page date shows today's date**	🟢 Low	The case header (line ~275) uses `new Date().toLocaleDateString()` instead of the report's `created_at` timestamp.
B7	**Admin sidebar hash links don't scroll**	🟢 Low	Admin sidebar uses `/admin#health`, `/admin#reports`, etc. These work for initial load but clicking them while already on `/admin` doesn't trigger smooth scrolling because Next.js client-side navigation doesn't scroll to hash anchors by default.

5.2 Architecture Concerns

#	Concern	Discussion
A1	**No state management library**	All state is component-local (`useState`) or React Context (Auth, Theme). This works for the current scale but may become unwieldy if the agent page grows (e.g., multi-turn conversations, multiple patient contexts, tool execution state). Consider whether Zustand or React Query would be beneficial.
A2	**API client is a singleton, not dependency-injected**	`apiClient` is a module-level singleton. This makes unit testing API-dependent components difficult — can't easily mock or replace it. Consider passing it via Context or accepting it as a hook parameter.
A3	**Hooks defined but unused**	Three hooks (`useCaseAnalysis`, `useReportApproval`, `useBackendStatus`) are defined in `hooks.ts` but no component uses them. Pages call `apiClient` methods directly. This creates two parallel patterns for the same action. Should either use hooks everywhere or remove them.
A4	**`src/lib/api.ts` (old file) vs `src/lib/api/` (new directory)**	The old monolithic `api.ts` was replaced with a structured `api/` directory. However, the old file may still exist on some developer machines if they haven't pulled clean. The barrel `api/index.ts` exists but nobody imports from `@/lib/api` — they all import from sub-modules (`@/lib/api/client`, `@/lib/api/types`). This inconsistency may confuse future developers.
A5	**No environment-based feature flags**	Whether auth is required, whether mock data is shown, etc., are all runtime-detected. Consider `.env` feature flags like `NEXT_PUBLIC_ENABLE_AUTH=true` to make behavior explicit.

5.3 UX / Design Gaps

#	Gap	Description
U1	**No patient selector in Co-Pilot**	The agent page always shows one hardcoded patient. Doctors need to select or search for a patient before starting an analysis.

#	Gap	Description
U2	**No conversation history / persistence**	Chat messages are lost on page reload. No conversation history, no save/load, no multi-session support.
U3	**No real-time feedback during analysis**	When the backend is online, the agent page shows a single "analyzing_text" status and then jumps to complete. The mock simulation shows a richer pipeline (routing → image → history → text → judging → report). The real API flow should show intermediate statuses.
U4	**No confirmation on destructive actions**	"Reject" on the approval bar immediately calls the API without a confirmation dialog.
U5	**No breadcrumb navigation**	Deep pages like `/case/report-001` or `/timeline/PT-12345` don't show breadcrumbs. Users rely on the back button or navbar.
U6	**Mobile experience on agent page**	The citations sidebar is 320px — on mobile, it's either hidden or takes the full width. The toggle exists but the panel overlay behavior could be improved.
U7	**No empty state for admin dashboard**	When backend is down, the admin page shows stat cards with "–" and a red "Offline" status. A more helpful empty state with setup instructions would be better.

5.4 Open Questions for Brainstorming

These are unresolved design and product decisions that need team discussion:

#	Question	Context
Q1	**Should auth be mandatory or optional?**	Currently, all pages except `/admin` are accessible without login. The agent page works without auth (it just won't inject the `Authorization` header). Should we require login to use the co-pilot? Or keep it open for demo purposes?
Q2	**How should medical images be handled?**	The `analyzeCase` API accepts `image_urls` (string URLs). Options: (a) upload to a backend file endpoint and get back a URL, (b) upload to S3/GCS and pass a presigned URL, (c) use base64 encoding in the request body. Backend team needs to clarify.
Q3	**What is the patient selection flow?**	Before using the co-pilot, does the doctor: (a) select from a patient list, (b) search by MRN, (c) start a new encounter that creates a patient? This affects the URL structure — should `/agent` become `/agent?patient=PT-12345`?

#	Question	Context
Q4	**Should the frontend support multi-turn conversations?**	Currently, each message triggers an independent `analyzeCase` call. Should we support context-aware multi-turn conversations where previous findings carry over? That would require a backend session/thread concept.
Q5	**How should we handle the judge verdict UI?**	The case page shows raw judge verdict data (conflicts, low-confidence items, missing context). Should this be a collapsible section, a modal, or inline with findings? What does the clinical user expect?
Q6	**What are the real admin requirements?**	The admin dashboard was built speculatively — health check, patient count. What does the team actually need? Audit logs? User management? Model performance metrics? We should define this before building out the stub sections.
Q7	**Do we need WebSocket / SSE for the agent page?**	The mock simulation shows a multi-step pipeline with intermediate statuses. If the backend supports streaming responses or status updates, we should integrate WebSocket/SSE. Otherwise, the frontend can only show a simple loading spinner during real API calls.
Q8	**Should the case page URL use report ID or encounter ID?**	Currently `/case/[id]` maps to a report ID. But the UX might be more natural with encounter IDs (one encounter can have multiple report versions).
Q9	**What role model do we need?**	Auth has 3 roles: `doctor`, `admin`, `nurse`. Only `admin` is gated. What should `nurse` see vs. `doctor`? Are there other roles (e.g., `radiologist`, `pathologist`)?
Q10	**Testing strategy?**	No tests exist. What should we prioritize: (a) unit tests with Vitest for hooks/utils, (b) component tests with Testing Library, (c) E2E tests with Playwright for critical user flows, (d) all of the above?

6. File-by-File Status Matrix

Pages

File	Status	API?	Mock Fallback?
`app/page.tsx`	✅ Complete	No	N/A
`app/agent/page.tsx`	✅ Functional	`analyzeCase`	✅ Yes

File	Status	API?	Mock Fallback?
`app/case/[id]/page.tsx`	✅ Functional	`getReport`, `getPatient`, `approveReport`	✅ Yes
`app/timeline/[patientId]/page.tsx`	✅ Functional	`getPatientTimeline`, `getPatient`	✅ Yes
`app/patients/page.tsx`	✅ Functional	`listPatients`, `createPatient`	❌ No
`app/admin/page.tsx`	⚠ Partial	`checkHealth`, `listPatients`	❌ No
`app/admin/layout.tsx`	✅ Complete	No	N/A
`app/auth/login/page.tsx`	✅ Complete	`login`	N/A
`app/auth/register/page.tsx`	✅ Complete	`register`	N/A
`app/error.tsx`	✅ Complete	No	N/A
`app/not-found.tsx`	✅ Complete	No	N/A

Components

File	Status	Notes
`agent/ChatArea.tsx`	✅ Complete	
`agent/ChatInput.tsx`	✅ Complete	Voice disabled
`agent/ChatMessage.tsx`	✅ Complete	Safe rendering
`agent/CitationsSidebar.tsx`	✅ Complete	Full ARIA
`agent/ToolProgress.tsx`	⚠ **Unused**	Not imported anywhere
`case/ImageViewer.tsx`	⚠ Placeholder	Emoji instead of `<img>`
`case/FindingsPanel.tsx`	✅ Complete	
`case/ReasoningTrace.tsx`	✅ Complete	
`case/ApprovalBar.tsx`	✅ Complete	
`landing/Hero.tsx`	✅ Complete	
`landing/Features.tsx`	✅ Complete	

File	Status	Notes
`landing/Architecture.tsx`	✅ Complete	
`landing/Stats.tsx`	✅ Complete	
`landing/CTA.tsx`	✅ Complete	
`landing/Footer.tsx`	✅ Complete	
`layout/Navbar.tsx`	✅ Complete	Auth-aware
`shared/ThemeToggle.tsx`	✅ Complete	
`shared/AgentStatusIndicator.tsx`	✅ Complete	
`shared/ErrorBoundary.tsx`	✅ Complete	
`shared/ProtectedRoute.tsx`	✅ Complete	
`shared/ConfidenceBadge.tsx`	✅ Complete	
`shared/SeverityBadge.tsx`	✅ Complete	
`shared/LoadingAnimation.tsx`	✅ Complete	

Library

File	Status	Notes
`lib/api/client.ts`	✅ Complete	Singleton, auth injection
`lib/api/types.ts`	✅ Complete	Full type coverage
`lib/api/index.ts`	✅ Complete	Barrel export
`lib/hooks.ts`	⚠ Partially used	3 of 7 hooks unused
`lib/types.ts`	✅ Complete	`Encounter` type unused
`lib/constants.ts`	⚠ Partially used	`API_PATHS` and `DEFAULTS` unused
`lib/utils.ts`	⚠ Partially used	`getConfidenceColor` unused
`lib/mock-data.ts`	✅ Complete	Coherent clinical scenario

---

7. Dead / Unused Code Inventory

Item	File	Line(s)	Action Suggested
`API_PATHS` constant object	`constants.ts`	10–23	Wire into `ApiClient` or remove
`DEFAULTS` constant object	`constants.ts`	46–55	Replace hardcoded `"PT-12345"` in Navbar or remove
`Encounter` interface	`types.ts`	12–19	Remove if not planned for use

Item	File	Line(s)	Action Suggested
`getConfidenceColor()`	`utils.ts`	~23	Remove (superseded by `getConfidenceBg`)
`useBackendStatus()` hook	`hooks.ts`	64	Use in agent page or remove
`useCaseAnalysis()` hook	`hooks.ts`	92	Use in agent page or remove
`useReportApproval()` hook	`hooks.ts`	126	Use in case page or remove
`ToolProgress` component	`agent/ToolProgress.tsx`	entire file	Use instead of inline tool rendering in ChatMessage, or remove

8. Hardcoded Values Inventory

Value	Location(s)	Should Be
`"PT-12345"`	Navbar link, mock-data.ts (8×)	`DEFAULTS.demoPatientId`
`"/case/demo"`	Navbar, Hero.tsx, CTA.tsx	`ROUTES.case("demo")`
`"/health"`, `"/patients"`, etc.	`api/client.ts` (11×)	`API_PATHS.*` from constants
`"report-001"`	`constants.ts` (defined, unused)	Reference where needed
`"MRN-2024-08812"`	`mock-data.ts`	Fine for mock data
`"/mock/cxr.png"`	`case/[id]/page.tsx` line 285	Dynamic from API response
`"en-US"`	`utils.ts` `formatDate` / `formatTime`	Browser locale
`25 * 1024 * 1024` (25 MB)	`constants.ts` (`DEFAULTS.maxUploadSize`)	Defined but no enforcement in ChatInput

End of report. Generated for internal team use — frontend workstream.